MODULE 10 · DOMAIN 2 - APPLICATIONS AND INTEGRATION

Designing the Application

One call or several, where code does the work, schemas between stages, session state, and retries that do no harm.

DOMAIN 2 of 8 EXAM WEIGHT 33.1% MODULE 10 of 36

admin@sthithapragna.com

A good design is an assembly line with **inspection stations**: each stage has one job, and nothing moves on until it passes.

admin@sthithapragna.com

One call, or several?

ONE CALL

Let Claude do the steps

- One task and one success criterion
- Intermediate steps need no inspection
- Adaptive thinking handles the reasoning
- Fewer calls, less latency, less to maintain

SEVERAL CALLS

Chain with checks

- Stages with distinct inputs, outputs and criteria
- You need to inspect, log or branch between steps
- Different models, effort or context per stage
- A failure must be traced to one stage

CURRENCY CHECK

Anthropic: Claude handles most multistep reasoning internally; chain when you need to inspect intermediate outputs or enforce a pipeline.

Copperlet's repair triage, as stages

01
Extract
Claude: ticket text to TicketFacts, structured output.

›

02
Check
Code: validate; emergency keywords force review.

›

03
Classify
Claude: urgency and trade as schema enums.

›

04
Route
Code: queue by urgency, trade and agency rules.

›

05
Draft
Claude: reply for an agent to approve.

THE TAKEAWAY

Three Claude stages, two code stages. Each Claude output is validated before the next stage runs.

Code or Claude?

GIVE IT TO CODE	GIVE IT TO CLAUDE
Arithmetic: totals, rent arrears, dates	Reading messy free text and documents
Lookups in your own systems	Classifying intent, urgency, topic
Exact business rules and thresholds	Summarising and drafting
Routing on known fields	Judgement on ambiguous input
Permissions and identity	Explaining results in plain language

THE TIEBREAKER

If the answer must be exact and the rule can be written down, write it in code. Claude reads, judges and writes.

Checks between stages

- **Validate the handoff:** schema, required fields, business rules, before the next stage runs.
- **Branch on the result:** continue, retry with the error, send to a person, or stop.
- **Self-correction chain:** draft, review against criteria, refine. Anthropic's most common chaining pattern.
- **Log every stage** with its input, output, prompt version and request ID.
- **A check that only logs** is not a gate. Gates decide what happens next.

> THE TAKEAWAY
>
> Every stage boundary is a chance to catch an error before it compounds.

Schemas as stage contracts

```python
class TicketFacts(BaseModel):                # stage 1 output: extraction
    ticket_id: str
    property_id: str
    problem: str
    vulnerable_tenant: bool | None           # None means "not stated"

class TriageDecision(BaseModel):             # stage 3 output: classification
    ticket_id: str                           # carried through every stage
    urgency: Urgency                         # enum: emergency, urgent, routine
    trade: Trade                             # enum of the seven trades
    schema_version: int = 2
```

5 Unknown is explicit, never a guess.

8, 11 One ID end to end, and a version so stages can change safely.

THE TAKEAWAY

A schema is the contract between two stages. Change it like an API: versioned and reviewed.

Sessions: your application holds the state

- **The Messages API is stateless.** Every request carries the history you choose to send.
- **Decide what a session holds:** messages, a summary, pinned facts, open actions.
- **Decide where it lives:** your server, keyed by a session ID, never trusted from the client.
- **Identity and permissions** come from authentication, never from the conversation.
- **Decide its life:** when it expires, what resets it, what carries over.

> ON THE EXAM
>
> Session design is a decision about what Claude sees next time. Make it on purpose.

Fresh session or carry on?

SITUATION	DECISION
Same task, same user, still active	Carry on, managing context as in module 8
Idle past the expiry window	Fresh session; carry over a short summary of open actions
User switches to a different account or record	Fresh session; nothing from the old record leaks across
A new, unrelated task	Fresh session, so old context cannot bias it
Facts may have changed since they were read	Re-fetch from the system of record, not from history

THE TIEBREAKER

History is a record of what was true then. For anything that changes, the system of record wins.

Retries that do no harm

- **Retrying a Claude call** costs tokens and may give a different answer, but changes nothing in the world.
- **Retrying a side effect** - a booking, a payment, a work order - can do it twice.
- **Idempotency key:** a stable ID for each action, checked and recorded before acting.
- **Save each stage's result,** so a retry resumes from the failed stage instead of starting again.
- **The SDKs retry some failures automatically,** so assume any step can run more than once.

> THE TAKEAWAY
>
> Design every step so that running it twice gives the same outcome as running it once.

Copperlet's tenant assistant sessions

WHAT A SESSION HOLDS

Kept on the server

- Session ID tied to the tenant's login
- Messages, plus the pinned tenancy block
- Compaction summary after 20 turns
- Open repairs and promises made

WHEN IT RESETS

Fresh by rule

- Idle for 24 hours
- Tenant switches to another tenancy
- Tenant logs out
- Rent and balances always re-fetched

THE SIGNAL

Identity comes from the login token. A tenant typing another tenant's name changes nothing.

> One prompt asks Claude to extract fields from a supplier invoice, compare them with the purchase order, decide whether to approve payment, and write an email to the supplier.
>
> Approval errors happen, and nobody can tell which part of the task went wrong.

What is the best redesign?

A Move the single prompt to the most capable model available

B Add more worked examples to the single prompt, covering each part of the task

C Raise the effort level, so that Claude reasons more about every part

D Split it into stages with their own outputs and checks, and compare against the purchase order in code

◆ PAUSE AND DECIDE

What is the best redesign?

A Move the single prompt to the most capable model available

Still one opaque step. Errors stay untraceable.

B Add more worked examples to the single prompt, covering each part of the task

A longer prompt, and the same blind spot.

C Raise the effort level, so that Claude reasons more about every part

More thinking, still nothing to inspect.

D Split it into stages with their own outputs and checks, and compare against the purchase order in code

Traceable stages, and exact matching done exactly.

THE SIGNAL

"Nobody can tell which part went wrong" plus an exact comparison: split the stages, and give the comparison to code.

A team classifies short product reviews as positive, neutral or negative. Their pipeline chains five calls: rephrase, analyse, classify, verify and format.

On their test set, a single call with a structured output scores the same. The pipeline is five times the cost and much slower.

What should the team do?

A Replace the pipeline with the single call, and keep checking it against the test set

B Add a sixth call that double-checks the verification step

C Run the five calls in parallel to reduce the latency

D Give each of the five calls a different model, to spread the work

◆ PAUSE AND DECIDE

What should the team do?

A Replace the pipeline with the single call, and keep checking it against the test set

Same quality, a fraction of the cost and latency.

B Add a sixth call that double-checks the verification step

More of the problem.

C Run the five calls in parallel to reduce the latency

Each step needs the last one's output.

D Give each of the five calls a different model, to spread the work

Complexity with no requirement behind it.

THE SIGNAL

"Scores the same" on a test set is the evidence. Chain only when stages earn their cost.

A letting agency's tool writes arrears letters. The prompt includes the tenant's ledger rows and asks Claude to total the amount owed and write a polite letter.

Now and then a letter states the wrong total.

What is the best fix?

A Tell Claude to check its arithmetic carefully before writing the letter

B Calculate the total in code, and give Claude the figure to put in the letter

C Set the effort level to maximum for every letter

D Add examples of correctly totalled ledgers to the prompt

◆ PAUSE AND DECIDE

What is the best fix?

A Tell Claude to check its arithmetic carefully before writing the letter

Guidance, not a guarantee.

B Calculate the total in code, and give Claude the figure to put in the letter

Exact work in code; Claude writes the words.

C Set the effort level to maximum for every letter

Lowers the error rate at a cost. Never to zero.

D Add examples of correctly totalled ledgers to the prompt

Examples shape format, not arithmetic.

THE SIGNAL

A money figure in a letter must be exact. Exact and writable as a rule means code.

> An insurer's pipeline extracts facts from a claim, then a second Claude call decides eligibility. The second stage often fails: field names vary between runs, and missing details are filled with plausible guesses.
>
> The team wants the handoff to be reliable.

Which two changes should they make? Choose two.

- A Define a strict schema for the first stage's output, enforced with structured outputs
- B Pass the first stage's whole conversation to the second stage so it can interpret
- C Make unknown values explicit nulls, and validate the handoff before the second stage runs
- D Merge both stages into a single call so there is no handoff
- E Tell the second stage to be flexible about field names

◆ PAUSE AND DECIDE

Which two changes should they make? Choose two.

A Define a strict schema for the first stage's output, enforced with structured outputs

Fixes the varying field names.

B Pass the first stage's whole conversation to the second stage so it can interpret

More context, same loose contract.

C Make unknown values explicit nulls, and validate the handoff before the second stage runs

Fixes the guesses, and gates the handoff.

D Merge both stages into a single call so there is no handoff

Loses the separate check on eligibility.

E Tell the second stage to be flexible about field names

Tolerates the bug instead of fixing it.

THE SIGNAL

Two symptoms, two fixes: varying names need a strict schema, guesses need explicit nulls and a gate.

A travel assistant keeps a single session per user indefinitely. Users who return weeks later get answers that mention old trips and out-of-date prices.

Within a single trip, the assistant works well.

What is the best fix?

A Move to a model with a larger context window, so that more history fits

B Tell Claude in the system prompt to ignore messages from earlier trips

C Define expiry and reset rules, carrying over only a short summary of current bookings, re-read from the booking system

D Delete the history after every message, so each turn starts clean

◆ PAUSE AND DECIDE

What is the best fix?

A Move to a model with a larger context window, so that more history fits

More stale history, not less.

B Tell Claude in the system prompt to ignore messages from earlier trips

Asks Claude to unsee what you chose to send.

C Define expiry and reset rules, carrying over only a short summary of current bookings, re-read from the booking system

Fresh sessions by rule, with current facts.

D Delete the history after every message, so each turn starts clean

Breaks the part that works.

THE SIGNAL

"Indefinitely" and "weeks later" are the signal: the session has no life cycle.

> A utility's assistant reads the customer's balance once at the start of a chat. Later in the same chat, a payment arrives, and the customer asks again.
>
> The assistant repeats the old balance.

What should the developer change?

A Add a line to the system prompt saying that balances can change

B Compact the conversation more often, so the old figure is summarised

C Shorten every session to a few turns

D Give Claude a tool that reads the balance from the billing system whenever it is needed

◆ PAUSE AND DECIDE

What should the developer change?

A. Add a line to the system prompt saying that balances can change

Claude still has only the old figure.

B. Compact the conversation more often, so the old figure is summarised

A summary of a stale number is still stale.

C. Shorten every session to a few turns

Hurts every conversation to dodge one bug.

D. Give Claude a tool that reads the balance from the billing system whenever it is needed

Volatile facts come from the system of record.

THE SIGNAL

A value that can change mid-session must be read when it is needed, not remembered.

A background worker takes each repair ticket, has Claude classify it, then calls a contractor's API to create a work order. If the contractor call times out after the order was created, the job is retried.

Contractors are receiving duplicate work orders.

What is the best fix?

A Turn off retries for the whole job

B Use the ticket ID as an idempotency key, recorded and checked before creating the order, and resume from the failed stage

C Ask Claude to check whether a work order already exists before classifying

D Increase the timeout on the contractor call

◆ PAUSE AND DECIDE

What is the best fix?

A. Turn off retries for the whole job
Trades duplicates for lost tickets.

B. Use the ticket ID as an idempotency key, recorded and checked before creating the order, and resume from the failed stage
Running twice now has the same outcome as once.

C. Ask Claude to check whether a work order already exists before classifying
A deterministic check, handed to a model.

D. Increase the timeout on the contractor call
Makes timeouts rarer, not harmless.

THE SIGNAL

Duplicates after a retry mean a side effect is not idempotent. Fix the step, not the retry.

A housing association's assistant decides whose records to look up from what the user types, such as "I'm the tenant at flat twelve". A tester reads another resident's details this way.

Users already sign in to the app before chatting.

What should the developer do?

A Have Claude ask security questions before it looks up any record

B Add a classifier that detects impersonation attempts in user messages

C Take identity from the signed-in session in code, and scope every lookup to that user's records

D Log every message that mentions another flat, for later review

◆ PAUSE AND DECIDE

What should the developer do?

A Have Claude ask security questions before it looks up any record

Puts access control in the prompt.

B Add a classifier that detects impersonation attempts in user messages

Detects some attempts; still trusts the chat.

C Take identity from the signed-in session in code, and scope every lookup to that user's records

Identity from authentication, enforced in code.

D Log every message that mentions another flat, for later review

Records the leak after it happens.

THE SIGNAL

"Users already sign in" is the signal: the identity is already known. Use it, in code.

Module 10 in five lines

- **One call** for one task; **several** when stages differ or must be inspected.
- **Code does exact work;** Claude reads, judges and writes.
- **Gate every handoff:** versioned schema, explicit nulls, one ID end to end.
- **Sessions are yours:** server-side, identity from login, reset rules, facts re-fetched.
- **Make every step idempotent,** so a retry resumes instead of repeating.

COMING UP

Next: how Claude reads instructions across surfaces, and content boundaries.